PrepPath E-books Store

DSA

Handwritten Notes

Data Structures & Algorithms
made easy for freshers

★ Hand-drawn diagrams & step-by-step logic

★ Python code, tested & ready to run

★ Coding-problem patterns for interviews

★ 30-day practice plan & web references

Learn it. Draw it. Code it.

START HERE

Contents & Roadmap

These notes explain **Data Structures & Algorithms (DSA)** the way a friend would on a whiteboard: simple words, a picture for every idea, and a tiny piece of code you can run. Follow the order below. Each chapter builds on the one before.



Learning order: left to right. Do not skip Big-O and Arrays!

Chapter	What you learn	Page
BIG-O	Speed, growth rates, space	3
Arrays	Basics, prefix sum, Kadane	6
Strings	Palindromes, anagrams	9
Windows	Two pointers, sliding window	11
Hashing	Maps, sets, counting	14
Recursion	Think small, call stack	16
Sorting	Bubble to quick sort	19
Binary Search	Halving the search	23
Linked List	Nodes, reverse, cycles	26
Stack	LIFO and monotonic stack	30
Queue	FIFO and deque	33
Trees	Traversals, BST	35
Heap	Priority queue, top K	40
Graphs	BFS, DFS, shortest path, DSU	42
Greedy	Best choice now	48
Backtracking	Try, undo, try again	50
Dynamic Programming	Remember sub-answers	52
Bits	XOR tricks	55
Problem Solving	Frameworks, patterns, plan	56

Study trick

How to study each page: 1) read the idea 2) cover the code and try to write it yourself 3) run it 4) solve the practice problems named on the page.

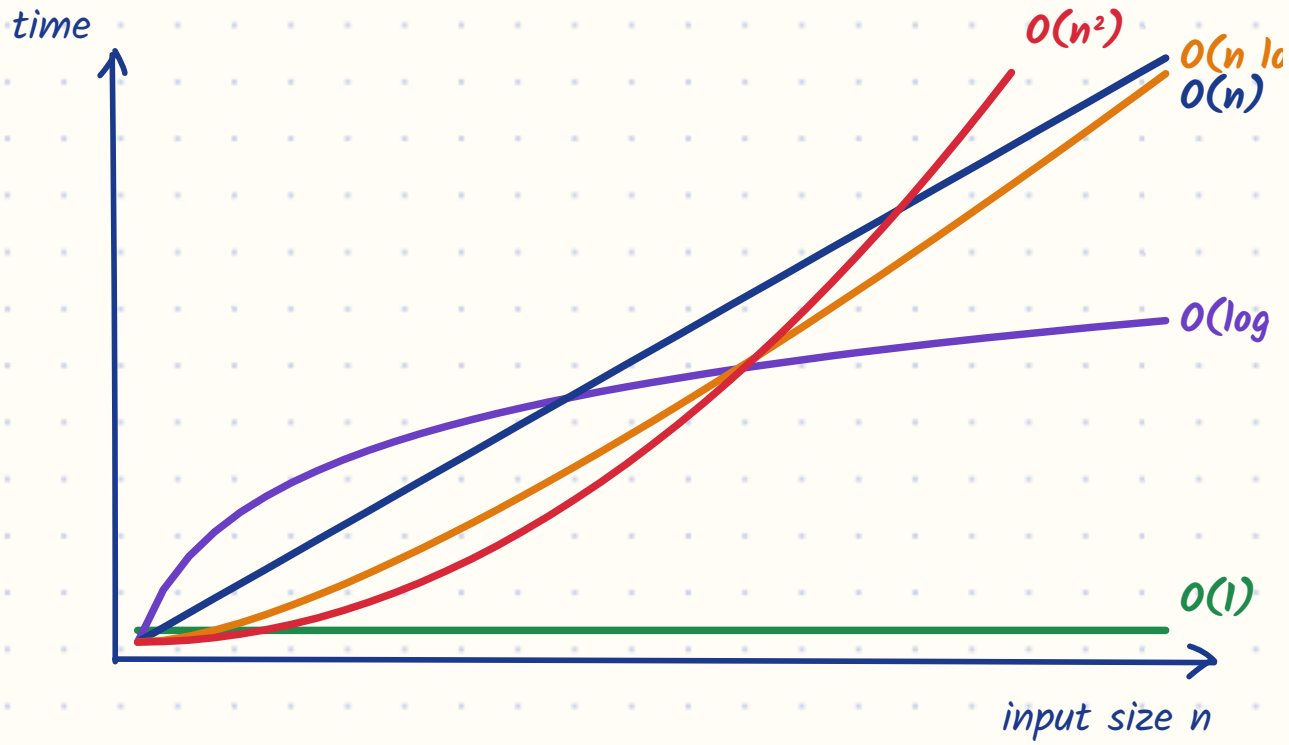
BIG-O

Big-O: How Fast Is Your Code?

Imagine two friends find a name in a phone book. One flips page by page. The other opens the middle and halves the search each time. Both get the answer, but one is ***much faster*** as the book grows. ***Big-O notation*** is the language we use to describe this.

The idea in one line

Big-O tells us ***how the number of steps grows when the input size*** `n` ***grows***. We ignore small details (like +5 steps) and keep only the fastest-growing part.



The shapes of the common growth rates. Flatter is faster.

Two golden rules

- ***Drop constants:*** `3n + 10` steps is just ***O(n)***.
- ***Keep the biggest term:*** `n² + n` is just ***O(n²)***.

> ***Remember***
> Big-O is usually the ***worst case*** — the slowest the code could be. It answers: *'Will this still work when n = 1,00,000?'*

BIG-O

Common Complexities

Big-O	Name	Example	n = 1,000,000 steps
O(1)	Constant	Read `arr[5]`	1
O(log n)	Logarithmic	Binary search	~20
O(n)	Linear	Loop through a list	1 million
O(n log n)	Linearithmic	Merge sort	~20 million
$O(n^2)$	Quadratic	Two nested loops	1 trillion (too slow!)
$O(2^n)$	Exponential	All subsets	Impossible

Quick feel for each

O(1) and O(n)

```
def first(a):              # O(1)
    return a[0]

def total(a):              # O(n)
    s = 0
    for x in a:
        s += x
    return s
```

O(n^2) and O(log n)

```
def pairs(a):              # O(n^2)
    c = 0
    for x in a:
        for y in a:
            c += 1
    return c

def halves(n):             # O(log n)
    c = 0
    while n > 1:
        n //= 2
        c += 1
    return c
```

Use this in contests

In coding rounds, an online judge allows about ***10^8 simple steps per second***. If n = 10^5, $O(n^2) = 10^{10}$ steps will time out, but O(n log n) will pass.

Cheat rule: $n \leq 10$ → anything works. $n \leq 1000$ → $O(n^2)$ is fine. $n \leq 10^5$ → aim for O(n log n). $n \leq 10^6$ → aim for O(n).

BIG-O

Analysing Code & Space

How to count steps

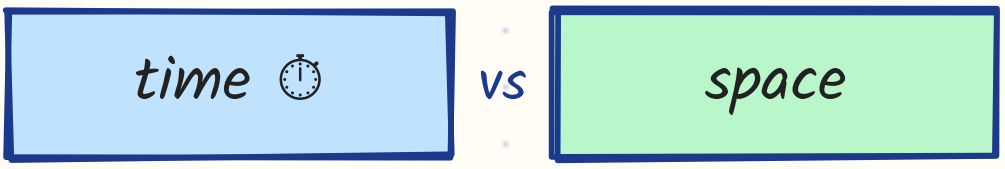
1. A single statement = O(1).
2. A loop that runs n times = O(n) × work inside.
3. Nested loops → multiply. Loops one after another → add, then keep the biggest.
4. A loop that halves/doubles each time → O(log n).

Example

```
def demo(a):
    n = len(a)
    for i in range(n):          # n
        for j in range(n):      # n   → n*n
            pass
    for k in range(n):          # n   → added
        pass
    return n
# total = n^2 + n  →  O(n^2)
```

Space complexity

Space complexity counts the *extra memory* your code needs, apart from the input. A few variables = O(1). A new list of size n = O(n). A recursion going n levels deep also uses O(n) stack memory.



Every solution has two costs. Often you can trade one for the other.

Big idea

Trade-off: a hash set uses O(n) extra space but makes lookups O(1). Many interview answers are just *'spend memory to save time'*.

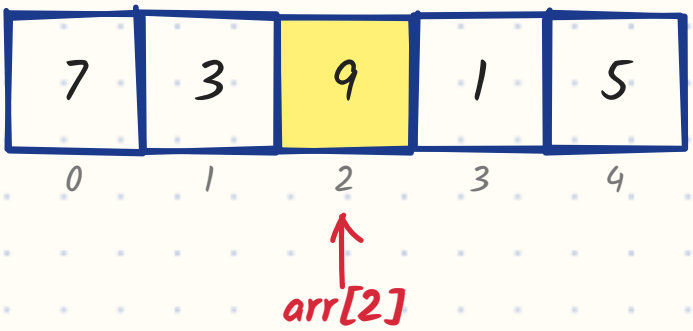
Practice

- What is the complexity of a loop `for i in range(0, n, 2)`? (still O(n))
- Complexity of `while n > 0: n //= 10`? (O(log n), digits of n)

ARRAYS

Arrays: The Foundation

An **array** stores items in **consecutive memory slots**, each with an index starting at **0**. Because the address is computed directly from the index, reading any element is instant.



Index starts at 0. arr[2] = 9, found in one step.

Operation	Time	Why
Read `arr[i]`	O(1)	direct address
Update `arr[i] = x`	O(1)	direct address
Search (unsorted)	O(n)	may check all
Insert / delete in middle	O(n)	shift elements
Append at end	O(1)*	amortized

Python lists in action

common list operations

```
a = [7, 3, 9, 1, 5]
a.append(4)        # add at end
a.insert(1, 8)     # add at index 1 (shifts others)
a.pop()            # remove last
a.remove(9)        # remove first 9
print(a, len(a), a[-1])   # a[-1] = last item
for i, x in enumerate(a): # index + value together
    pass
```

Careful

Inserting at the **front** is slow (O(n)) because every element shifts right. Prefer adding at the end.

Traversal patterns

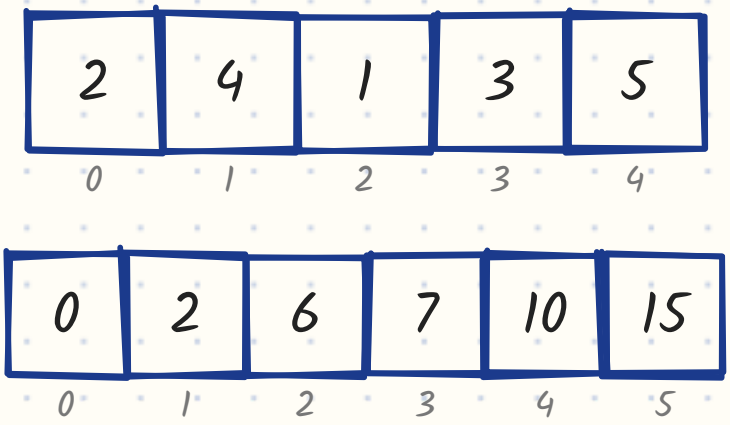
- Forward: `for x in a`
- Backward: `for i in range(len(a)-1, -1, -1)`
- Two ends: `i = 0, j = n-1`, move toward each other

ARRAYS

Prefix Sums & Kadane

Prefix sum: answer range sums fast

Question: *sum of elements from index L to R*, asked many times. Adding each time = O(n) per query. Instead build a ***prefix array*** once: `pre[i]` = sum of first i items. Then ***sum(L..R) = pre[R+1] − pre[L]***, in O(1).



Top: array. Bottom: prefix (starts with 0). Sum of index 1..3 = 10 − 2 = 8

prefix sum

```
def build(a):
    pre = [0]
    for x in a:
        pre.append(pre[-1] + x)
    return pre

def range_sum(pre, L, R):      # inclusive
    return pre[R + 1] - pre[L]

pre = build([2, 4, 1, 3, 5])
print(range_sum(pre, 1, 3))    # 8
```

Kadane: maximum subarray sum

Find the contiguous part with the biggest sum, e.g. `[-2,1,-3,4,-1,2,1,-5,4]` → `[4,-1,2,1]` = **6**. Idea: walk once; at each item either ***extend*** the running sum or ***restart*** from the item (if the running sum went negative).

Kadane O(n)

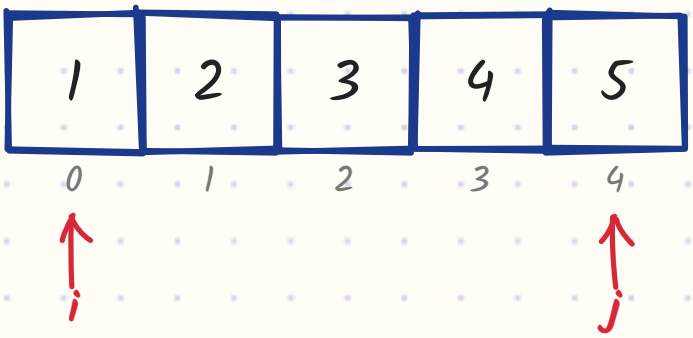
```
def max_subarray(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)   # extend or restart
        best = max(best, cur)
    return best
```

Array Problems: Think, Then Code

ARRAYS

1. Reverse an array in place

Swap the two ends and move inward. No extra list needed: **O(n) time, O(1) space**.



swap(a[i], a[j]); i++; j--

```
def reverse(a):
    i, j = 0, len(a) - 1
    while i < j:
        a[i], a[j] = a[j], a[i]
        i += 1; j -= 1
    return a
```

2. Rotate right by k

Trick: **reverse all, then reverse first k, then reverse the rest**. Remember `k = k % n`.

```
def rotate(a, k):
    n = len(a); k %= n
    def rev(l, r):
        while l < r:
            a[l], a[r] = a[r], a[l]; l += 1; r -= 1
    rev(0, n-1); rev(0, k-1); rev(k, n-1)
    return a
```

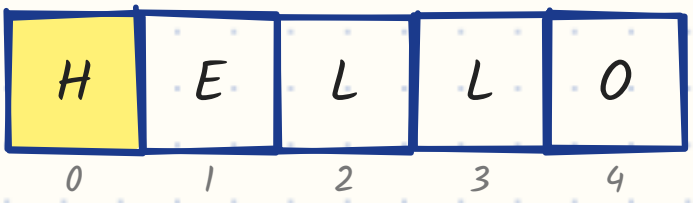
Try these

Practice: Second largest element · Move zeros to the end (use a write pointer) · Merge two sorted arrays · Find the missing number in 1..n (sum formula n(n+1)/2).

STRINGS

Strings

A ***string*** is just an array of characters. In Python, strings are ***immutable*** — you cannot change one character; you build a new string.



s[0] = 'H', s[-1] = 'O', len(s) = 5

Handy operations

```
s = "Hello World"
s.lower(); s.upper()          # case
s.split()                     # ['Hello', 'World']
" ".join(["a", "b"])          # 'a b'
s[::-1]                       # reverse → 'dlroW olleH'
s.find("World")               # 6 (or -1)
s.isalnum(); s.isdigit()      # character checks
ord('a'), chr(97)             # ASCII ⟷ char
```

Speed tip

Repeated `s = s + ch` inside a loop copies the string each time ($O(n^2)$). Collect pieces in a ***list*** and `''.join( ... )` at the end.

Palindrome check

A palindrome reads the same both ways. Use two pointers from the ends, skipping non-letters.

```
def is_palindrome(s):
    i, j = 0, len(s) - 1
    while i < j:
        while i < j and not s[i].isalnum(): i += 1
        while i < j and not s[j].isalnum(): j -= 1
        if s[i].lower() ≠ s[j].lower(): return False
        i += 1; j -= 1
    return True
```

STRINGS

String Problems You Must Know

Valid anagram

Two words are anagrams if they use the **same letters with the same counts**.

Count letters with a dictionary.

```
from collections import Counter
def is_anagram(a, b):
    return Counter(a) == Counter(b)
```

First non-repeating character

```
def first_unique(s):
    cnt = Counter(s)
    for i, ch in enumerate(s):
        if cnt[ch] == 1:
            return i
    return -1
```

Reverse words in a sentence

```
def reverse_words(s):
    return " ".join(s.split()[::-1])
```

Longest common prefix

Compare character by character down the list. Stop at the first mismatch.

```
def common_prefix(words):
    pre = words[0]
    for w in words[1:]:
        while not w.startswith(pre):
            pre = pre[:-1]
    return pre
```

Pattern spotter

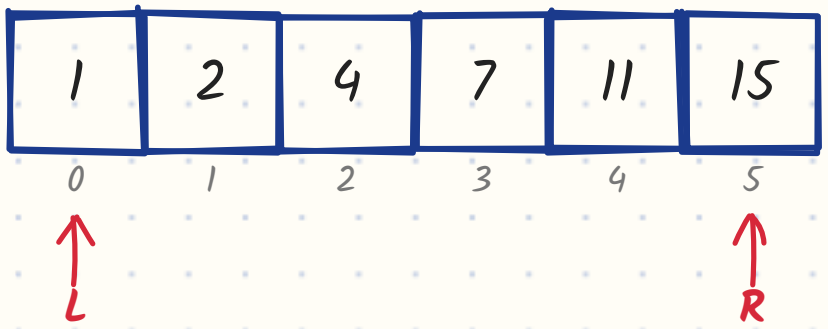
Interview pattern: string problem + counting → **hash map**. String problem + ends/palindrome → **two pointers**. Substring problem → **sliding window** (coming soon).

WINDOWS

Two Pointers

Instead of two nested loops ($O(n^2)$), use **two indices** that move through the data. It works best on **sorted arrays** or when you compare the two ends.

Pattern A: opposite ends



Target sum = 15. 1+15 = 16 is too big, so move R left.

If the sum is **too big**, move `R` left. If **too small**, move `L` right. Each step discards one element, so it is **O(n)**.

```
def two_sum_sorted(a, target):
    L, R = 0, len(a) - 1
    while L < R:
        s = a[L] + a[R]
        if s == target: return [L, R]
        if s < target: L += 1
        else: R -= 1
    return []
```

Pattern B: slow and fast

Both start at the left. `fast` scans, `slow` marks where good items go. Example: remove duplicates from a sorted array.

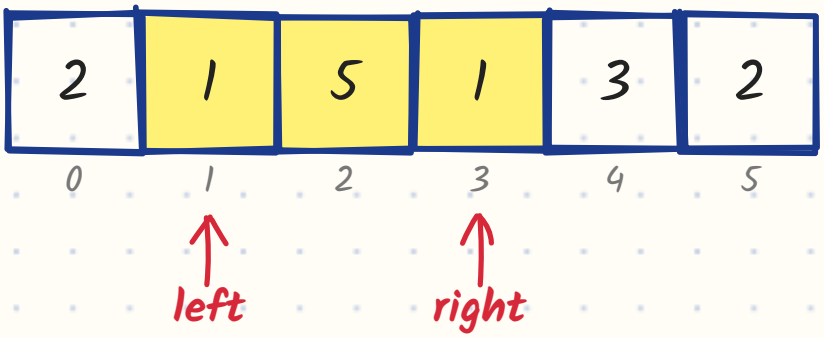
```
def remove_duplicates(a):
    if not a: return 0
    slow = 0
    for fast in range(1, len(a)):
        if a[fast] ≠ a[slow]:
            slow += 1
            a[slow] = a[fast]
    return slow + 1        # new length
```

Level up
Container with most water, 3Sum, trapping rain water are famous two-pointer problems. Sort first, then fix one number and use two pointers for the rest.

WINDOWS

Sliding Window: Fixed

A **window** is a block of consecutive items. Instead of recalculating the whole block each time, **slide** it: add the new item on the right, remove the old item on the left. This turns O(n·k) into **O(n)**.



Window size k = 3. Sum = 1+5+1 = 7. Next: add 3, remove 2 → 8.

Problem: max sum of k consecutive items

fixed window

```
def max_sum_k(a, k):
    cur = sum(a[:k])              # first window
    best = cur
    for i in range(k, len(a)):
        cur += a[i] - a[i - k]    # add new, drop old
        best = max(best, cur)
    return best
```

When to use

Use a fixed window when the question says **'subarray of size k'**: averages, maximum sum, count of distinct items in every window of size k.

Why is it fast?

- Each element **enters** the window once and **leaves** once.
- So total work is about 2n steps = **O(n)**.
- The slow way recomputes k items for every position.

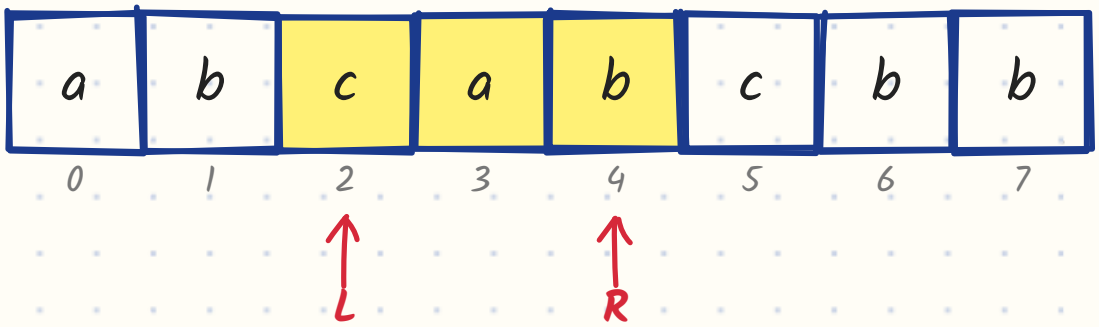
Practice

- Average of all subarrays of size k
- Count windows of size k with at least one vowel

WINDOWS

Sliding Window: Variable

Sometimes the window size is not fixed. You **expand** `right` to include more, and **shrink** `left` whenever the window breaks a rule. Use it for 'longest/shortest substring with ...'.



Longest substring without repeating letters: 'abc' has length 3

no repeating characters

```
def longest_unique(s):
    seen = {}                  # char → last index
    left = best = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] ⩾ left:
            left = seen[ch] + 1       # shrink past the repeat
        seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

Template to remember

template (pseudo-code)

```
left = 0
for right in range(len(a)):
    add a[right] to the window
    while window is invalid:
        remove a[left] from the window
        left += 1
    update the answer (right - left + 1)
```

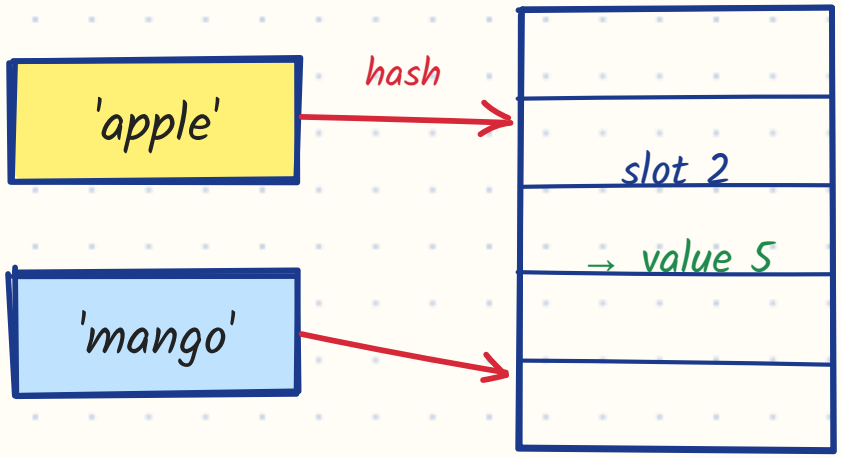
Classic

Smallest subarray with sum ≥ target: expand right adding numbers; while sum ≥ target, record length and shrink left. Works only when numbers are **positive**.

HASHING

Hashing: Instant Lookup

A **hash map** (dictionary) stores **key → value** pairs. A hash function turns the key into an index so lookups take about **O(1)**. A **hash set** stores only keys and answers *'have I seen this?'* instantly.



Key → hash function → index → value

Operation	Dict / Set	List
Insert	O(1)	O(1) at end
Lookup / `in`	O(1)	O(n)
Delete	O(1)	O(n)

Python usage

```
d = {}                      # dictionary
d["apple"] = 3              # insert
d["apple"] += 1             # update
d.get("kiwi", 0)            # safe read → 0
"apple" in d                # True
seen = set([1, 2, 3])       # set
seen.add(4); 2 in seen      # True
from collections import Counter, defaultdict
Counter("banana")           # {'a':3,'n':2,'b':1}
```

Golden trick

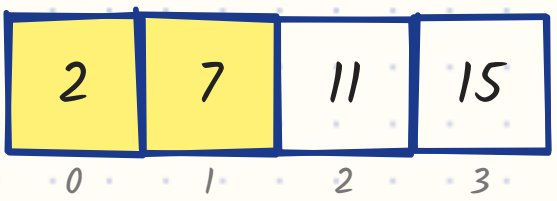
If you catch yourself writing a loop **inside a loop just to search**, replace the inner loop with a **hash lookup**. It often turns O(n²) into O(n).

HASHING

Hashing Problems

1. Two Sum (the most famous problem)

Find two numbers that add up to the target. For each `x`, the partner we need is `target - x`. Store seen numbers in a dict: `value → index`.



target = 9 → need 7 when at 2. Found at the next step.

```
def two_sum(a, target):
    pos = {}
    for i, x in enumerate(a):
        need = target - x
        if need in pos:
            return [pos[need], i]
        pos[x] = i
    return []
```

2. Group anagrams

Words that are anagrams have the same ***sorted letters***. Use the sorted word as the dictionary key.

```
from collections import defaultdict
def group_anagrams(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())
```

3. Subarray sum equals k

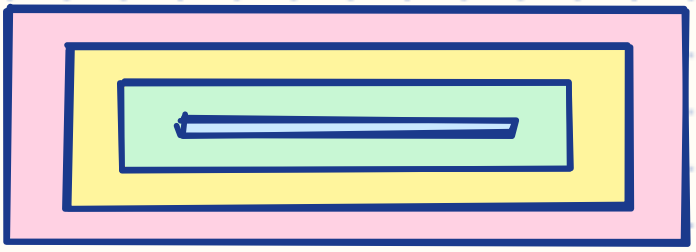
Use prefix sums + dict: if `prefix - k` was seen before, a subarray ending here sums to k.

```
def subarray_sum(a, k):
    cnt = {0: 1}; pre = ans = 0
    for x in a:
        pre += x
        ans += cnt.get(pre - k, 0)
        cnt[pre] = cnt.get(pre, 0) + 1
    return ans
```

RECURSION

Recursion: Think Small

Recursion means a function **calls itself** on a smaller version of the same problem. Think of Russian nesting dolls: open one, find a smaller one, until the tiny solid doll at the centre.



smaller and smaller → base case

Each call works on a smaller problem.

Every recursion needs two things

- **Base case:** the smallest problem, answered directly. It **stops** the recursion.
- **Recursive case:** reduce the problem and call yourself.

two classics

```
def factorial(n):
    if n ⩽ 1:                  # base case
        return 1
    return n * factorial(n-1)  # recursive case

def sum_to(n):                 # 1 + 2 + ... + n
    if n == 0: return 0
    return n + sum_to(n - 1)
```

Danger

No base case = infinite calls = crash (`RecursionError` in Python, stack overflow elsewhere). Always write the base case first!

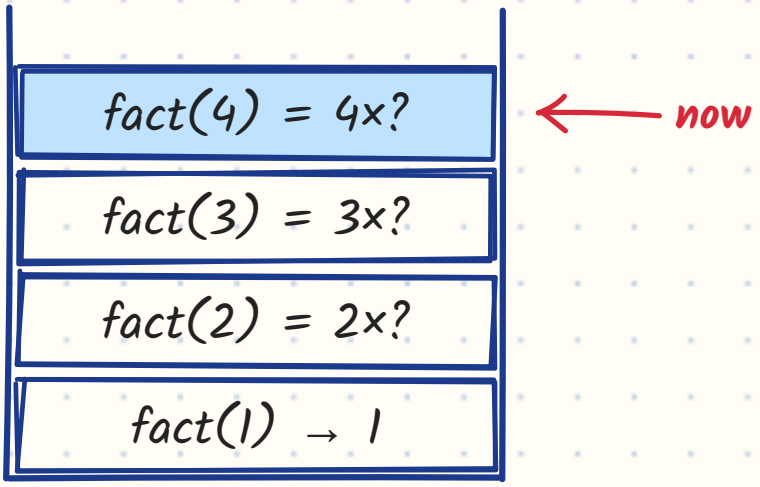
Three-step recipe

1. Say in words: *'The answer for n equals … the answer for n−1'*.
2. Write the **base case** .
3. Trust the smaller call; combine its result.

RECURSION

The Call Stack

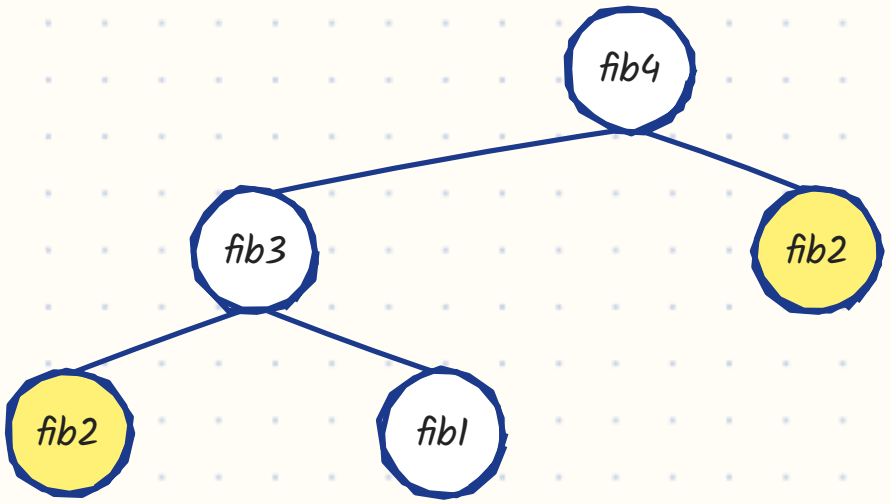
Each call waits on the ***call stack*** until the call it made returns. Here is `factorial(4)`: calls pile up going down, then answers return coming back up.



Stack grows until the base case, then unwinds: 1 → 2 → 6 → 24

Fibonacci and the repeated-work problem

`fib(n) = fib(n-1) + fib(n-2)`. Simple, but notice the same values get recomputed again and again.



fib2 is calculated again and again. For big n this explodes to $O(2^n)$.

memoization: remember answers → O(n)

```
def fib(n, memo={}):
    if n ⩽ 1: return n
    if n in memo: return memo[n]          # reuse the answer
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

RECURSION

Recursion Problems

1. Power: x to the n

Halving the exponent makes it **O(log n)**: `x^n = (x^(n/2))²` (times x if n is odd).

```
def power(x, n):
    if n == 0: return 1
    half = power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x
```

2. Reverse a string

```
def rev(s):
    if len(s) <= 1: return s
    return rev(s[1:]) + s[0]
```

3. Palindrome check

```
def is_pal(s):
    if len(s) < 2: return True
    return s[0] == s[-1] and is_pal(s[1:-1])
```

4. Sum of digits

```
def digit_sum(n):
    return 0 if n == 0 else n % 10 + digit_sum(n // 10)
```

5. Tower of Hanoi (moves count)

Move n−1 disks away, move the big one, move n−1 back: `T(n) = 2·T(n-1) + 1` → **$2^n - 1$** moves.

```
def hanoi(n, a="A", b="B", c="C"):
    if n == 0: return 0
    return hanoi(n-1, a, c, b) + 1 + hanoi(n-1, c, b, a)
```

Why it matters

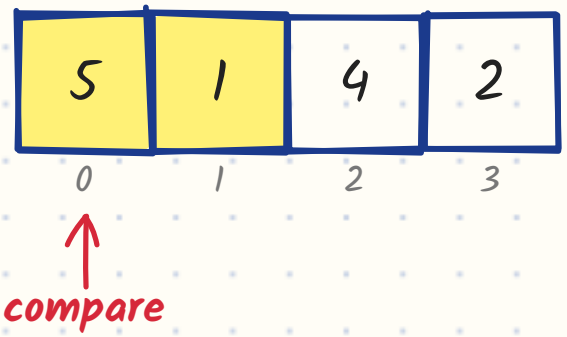
Recursion is the backbone of **trees, backtracking and DP**. Master it here and those chapters become easy.

SORTING

Sorting: Overview & Basic Sorts

Sorting puts items in order (smallest to largest). It makes searching faster and many problems simpler. Start with three easy sorts that all take $O(n^2)$.

Bubble sort: big values float to the end



Compare neighbours; swap if the left one is bigger. After pass 1, the largest is at the end.

```
def bubble(a):
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                swapped = True
        if not swapped: break      # already sorted
    return a
```

Selection sort repeatedly picks the minimum of the unsorted part and swaps it to the front (also $O(n^2)$).

Insertion sort: sort a hand of cards

```
def insertion(a):
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j ⩾ 0 and a[j] > key:
            a[j+1] = a[j]; j -= 1
        a[j+1] = key
    return a
```

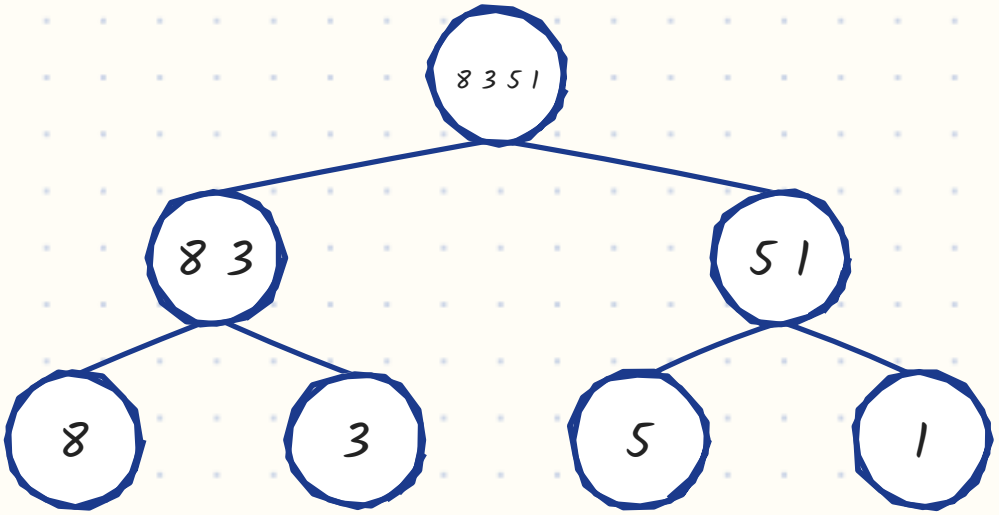
Fun fact

Insertion sort is **fast on nearly-sorted data** ($O(n)$) and is used inside real libraries for small arrays.

SORTING

Merge Sort: Divide & Conquer

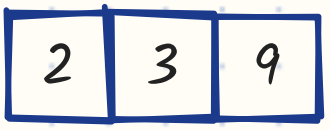
Divide the array into halves, **sort** each half (recursively), then **merge** two sorted halves into one. Time: **O(n log n)** always. Extra space: O(n).



Split until single items, then merge back up in order.

Merging two sorted lists





Always take the smaller front item. Both lists are already sorted.

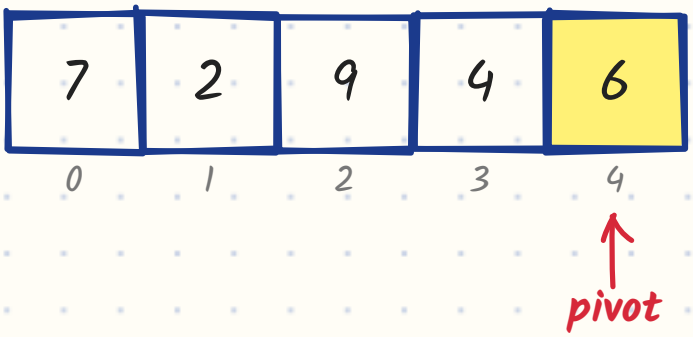
```
def merge_sort(a):
    if len(a) ⩽ 1: return a
    mid = len(a) // 2
    L, R = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] ⩽ R[j]: out.append(L[i]); i += 1
        else:           out.append(R[j]); j += 1
    return out + L[i:] + R[j:]
```

Why O(n log n)? The array is halved **log n** times, and every level does **n** work to merge.

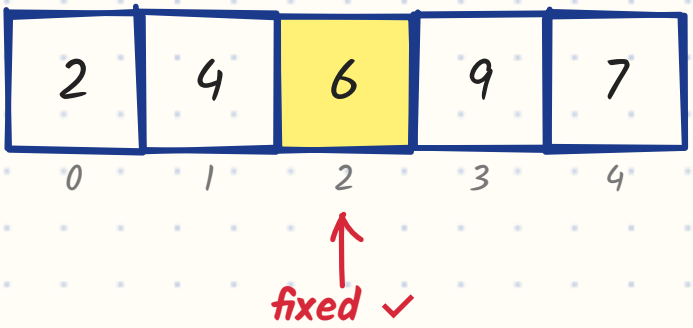
SORTING

Quick Sort

Pick a ***pivot***. Put smaller items to its left and larger to its right. Now the pivot is in its ***final place***. Repeat on both sides. Average ***O(n log n)***, worst case O(n²) with a bad pivot.



Pivot = 6. Smaller: 2, 4. Larger: 7, 9.



After partition, 6 is in the right place.

```
def quick_sort(a, lo=0, hi=None):
    if hi is None: hi = len(a) - 1
    if lo < hi:
        p = partition(a, lo, hi)
        quick_sort(a, lo, p - 1)
        quick_sort(a, p + 1, hi)
    return a

def partition(a, lo, hi):
    pivot, i = a[hi], lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]; i += 1
    a[i], a[hi] = a[hi], a[i]
    return i
```

Tip

Choosing a ***random pivot*** (or median of three) avoids the O(n²) worst case on already-sorted input.

SORTING

Sorting Cheat Sheet

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes

Which one should I use?

- **In real code:** just call `sorted(a)` / `a.sort()` — it is Timsort, O(n log n).
- **Tiny or nearly sorted data:** insertion sort.
- **Need guaranteed speed + stability:** merge sort.
- **Numbers in a small range:** counting sort (O(n)).
- **Interview:** be ready to code merge sort and quick sort.

Sorting with custom rules in Python

```
people = [("Asha", 22), ("Ravi", 19), ("Meena", 22)]
people.sort(key=lambda p: (-p[1], p[0]))   # age desc, name asc
words = sorted(["bb", "a", "ccc"], key=len)  # by length
```

Stable?

Stable means equal items keep their original relative order. This matters when you sort by one column and then another.

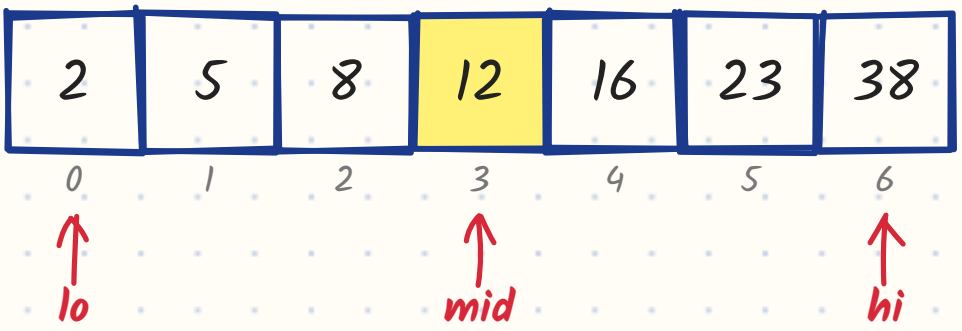
Problems that become easy after sorting

- Find duplicates (neighbours are equal)
- Merge overlapping intervals
- Two pointers on sorted data (3Sum)
- Closest pair / minimum difference

BINARY SEARCH

Binary Search

To find a name in a sorted phone book you open the middle: too far? go left. Too early? go right. ***Each step throws away half.*** For 1,000,000 items that is only about ***20 steps***: ***O(log n)***. It works only on ***sorted*** data.



Find 23: mid = 12 < 23 → search the right half.

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid + 1
        else:               hi = mid - 1
    return -1
```

Bug alert
In languages with fixed-size integers write `mid = lo + (hi - lo) // 2` to avoid overflow. Python is safe, but good habit.

Three things people get wrong

- Loop condition: `lo ⩽ hi` (not `<`).
- Updating with `mid + 1` / `mid - 1` (otherwise infinite loop).
- Using it on ***unsorted*** data.

Python has it built in: `bisect.bisect_left(a, x)` gives the insertion position.

BINARY SEARCH

Binary Search Variations

First and last position of a value

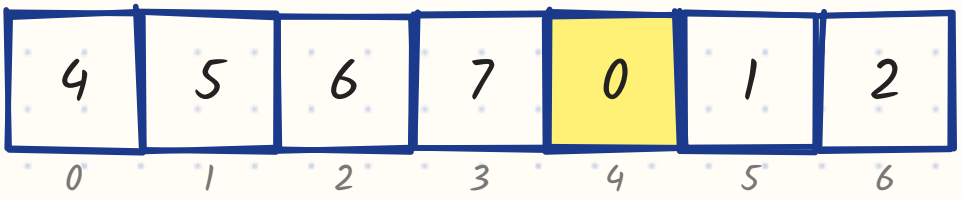
When duplicates exist, keep searching **after a hit** to find the boundary.

first occurrence

```
def first_pos(a, t):
    lo, hi, ans = 0, len(a) - 1, -1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if a[mid] == t: ans = mid; hi = mid - 1   # keep going left
        elif a[mid] < t: lo = mid + 1
        else: hi = mid - 1
    return ans
```

For the **last** position, on a hit set `lo = mid + 1` instead.

Search in a rotated sorted array



Sorted array rotated. One half is always sorted — decide which half holds the target.

```
def search_rotated(a, t):
    lo, hi = 0, len(a) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if a[mid] == t: return mid
        if a[lo] ⩽ a[mid]:                  # left half sorted
            if a[lo] ⩽ t < a[mid]: hi = mid - 1
            else: lo = mid + 1
        else:                               # right half sorted
            if a[mid] < t ⩽ a[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
```

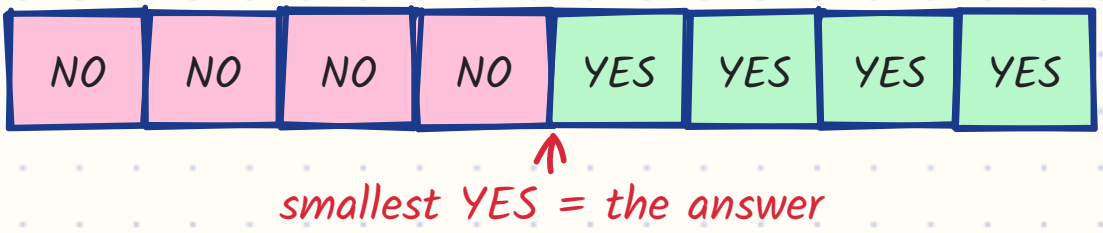
Practice

Also try: **search insert position**, **peak element**, **minimum in rotated array**, **square root of n** (binary search on 0..n).

BINARY SEARCH

Binary Search on the Answer

A powerful trick: even when there is **no sorted array**, you can binary search the **answer itself**, if the answers go from 'impossible' to 'possible' in one switch (a monotonic yes/no).



Check feasibility for a guess, then move left or right.

Example: Koko eats bananas

Piles of bananas, `h` hours. Find the **minimum eating speed** `k` (bananas/hour) so all piles finish in `h` hours. Speed from 1 to max(pile). If speed `k` works, any larger speed works too. So binary search `k`.

```
import math
def min_speed(piles, h):
    def hours(k): return sum(math.ceil(p / k) for p in piles)
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if hours(mid) ⩽ h: hi = mid       # works → try smaller
        else: lo = mid + 1
    return lo
```

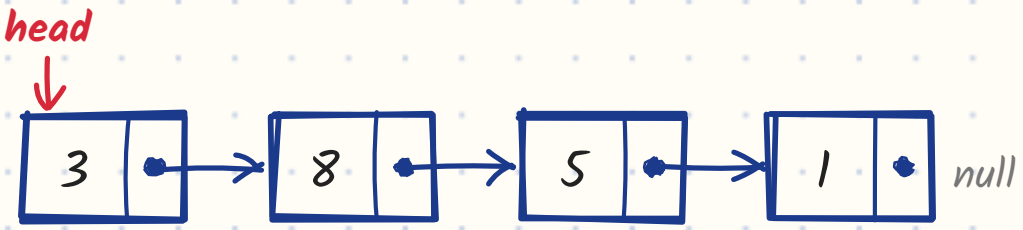
Template

Template: `lo` / `hi` = range of possible answers → write `feasible(x)` → if feasible go left (`hi = mid`) else go right (`lo = mid + 1`). Other problems: **ship packages in D days, split array largest sum, aggressive cows.**

LINKED LIST

Linked List Basics

A **linked list** is a chain of **nodes**. Each node holds a **value** and a **pointer to the next node**. Nodes can sit anywhere in memory; the pointers connect them. The last node points to `null` (`None`).



head points to the first node. Follow `next` to walk the list.

Operation	Array	Linked list
Access i-th item	O(1)	O(n)
Insert / delete at front	O(n)	O(1)
Insert after a known node	O(n)	O(1)
Search	O(n)	O(n)

node class + traversal

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

head = Node(3, Node(8, Node(5, Node(1))))
cur = head
while cur:                    # traversal
    print(cur.val)
    cur = cur.next
```

Trade-off

There is **no index** in a linked list. To reach the 4th node you must walk 3 steps. In exchange, inserting and deleting needs **no shifting**.

- **Singly** linked: `next` only.
- **Doubly** linked: `next` and `prev` (can go back).
- **Circular**: last node points back to the first.

LINKED LIST

Insert, Delete, Reverse

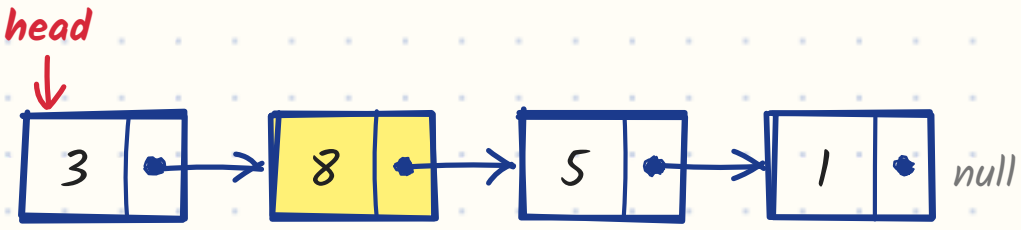
Insert at the front

O(1)

```
def push_front(head, v):
    return Node(v, head)          # new node becomes the head
```

Delete a value

To delete a node, make the ***previous node skip it***: `prev.next = prev.next.next`. A ***dummy node*** in front removes the special case for the head.



Delete 8: change 3's next to point to 5.

```
def delete(head, target):
    dummy = Node(0, head)
    prev = dummy
    while prev.next:
        if prev.next.val == target:
            prev.next = prev.next.next
        else:
            prev = prev.next
    return dummy.next
```

Reverse a linked list (very popular!)

Walk once and flip every pointer. Keep three names: `prev`, `cur`, `nxt`.

```
def reverse(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next        # 1. remember the rest
        cur.next = prev       # 2. flip the arrow
        prev, cur = cur, nxt  # 3. step forward
    return prev
```

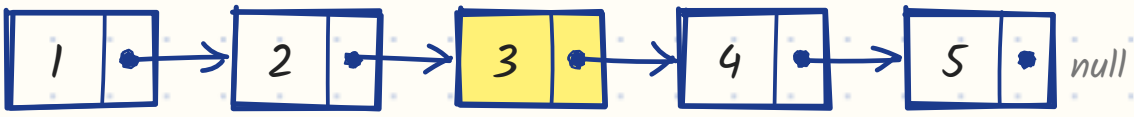
LINKED LIST

Fast & Slow Pointers

Two pointers move at different speeds: ***slow*** moves 1 step, ***fast*** moves 2. This 'tortoise and hare' trick solves many list problems with ***O(1) space***.

1. Find the middle

When `fast` reaches the end, `slow` is at the middle.



Slow stops at 3 when fast falls off the end.

```
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

2. Detect a cycle (Floyd's algorithm)

If the list loops, fast eventually ***laps*** slow and they meet. If there is no loop, fast reaches `None`.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

Another use

Remove n-th node from the end: move `fast` n steps ahead first, then move both together. When fast reaches the end, slow is just before the target.

LINKED LIST

Linked List Problems

Merge two sorted lists

Use a dummy head and always attach the smaller front node.

```
def merge_two(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.val ⩽ b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b          # attach whatever is left
    return dummy.next
```

Palindrome linked list

Find the middle, reverse the second half, compare both halves.

```
def is_pal_list(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    prev = None
    while slow:                          # reverse second half
        slow.next, prev, slow = prev, slow, slow.next
    a, b = head, prev
    while b:
        if a.val ≠ b.val: return False
        a, b = a.next, b.next
    return True
```

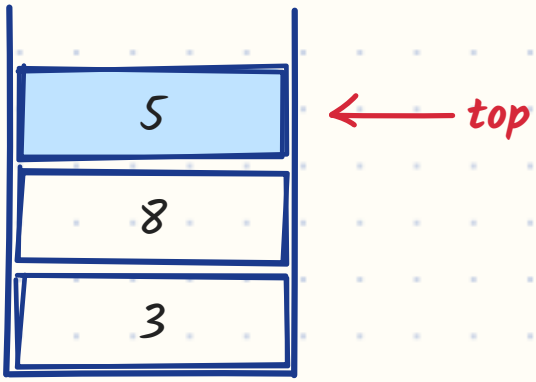
Add two numbers (digits stored in reverse)

```
def add_lists(a, b):
    dummy = tail = Node(0); carry = 0
    while a or b or carry:
        s = carry + (a.val if a else 0) + (b.val if b else 0)
        carry, d = divmod(s, 10)
        tail.next = Node(d); tail = tail.next
        a = a.next if a else None; b = b.next if b else None
    return dummy.next
```

Before you code
Linked-list checklist: draw boxes and arrows · use a ***dummy node*** · watch for `None` · save `next` ***before*** you change it.

STACK Stack: Last In, First Out

A **stack** is like a pile of plates: you add (**push**) and remove (**pop**) only at the **top**. The **last** item in is the **first** out — **LIFO**.



push(5) lands on top. pop() removes 5 first.

Operation	Meaning	Time
push(x)	add on top	O(1)
pop()	remove top	O(1)
peek()	look at top	O(1)
is_empty()	no items?	O(1)

```
stack = []
stack.append(3)      # push
stack.append(8)
top = stack[-1]      # peek → 8
stack.pop()          # pop  → 8
print(len(stack) == 0)
```

Where stacks appear

- **Undo** in editors; browser **back** button
- **Function calls** (the call stack!)
- **Matching brackets** and expression evaluation
- **DFS** (depth-first search) and backtracking

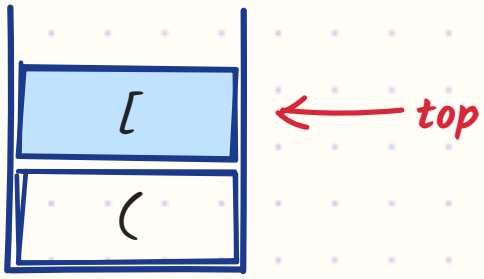
Python tip
In Python use a **list** as a stack (`append` / `pop` are O(1) at the end). Never use `pop(0)` — that is O(n).

STACK

Stack Problems

1. Valid parentheses

Push every opening bracket. For a closing bracket, the top must be its partner. At the end the stack must be empty.



Input '([)]'? The ')' meets '[' on top → invalid.

```
def valid(s):
    pair = {")": "(", "]": "[", "}": "{"}
    st = []
    for ch in s:
        if ch in pair:
            if not st or st.pop() ≠ pair[ch]:
                return False
        else:
            st.append(ch)
    return not st
```

2. Evaluate postfix (RPN)

Numbers are pushed. An operator pops two numbers, computes, and pushes the result.

```
def eval_rpn(tokens):
    st = []
    for t in tokens:
        if t in "+-*/" and len(t) == 1:
            b, a = st.pop(), st.pop()
            st.append(a + b if t == "+" else a - b if t == "-"
                      else a * b if t == "*" else int(a / b))
        else:
            st.append(int(t))
    return st[0]
```

Bonus

Min stack idea: store pairs `(value, minimum so far)` to read the minimum in O(1).

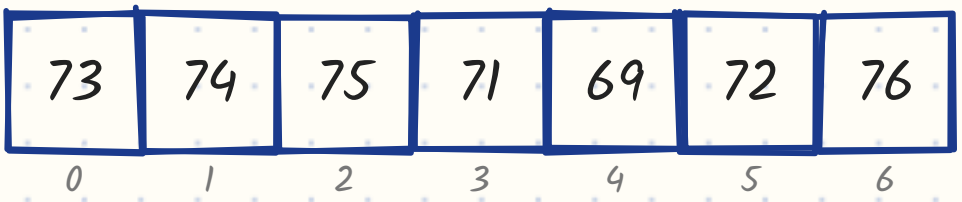
STACK

Monotonic Stack

A ***monotonic stack*** keeps its items in increasing or decreasing order. Before pushing, ***pop everything that breaks the order***. It solves 'next greater element' problems in ***O(n)***.

Next greater element

For each day, how many days until a warmer day? Keep a stack of ***indices*** with waiting (cooler) temperatures. A warmer day resolves them.



Day 0 (73) waits 1 day for 74. Day 2 (75) waits 4 days for 76.

```
def daily_temps(t):
    ans = [0] * len(t)
    st = []                          # indices, temps decreasing
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop()
            ans[j] = i - j           # found the warmer day
        st.append(i)
    return ans
```

Why O(n)?

Each index is pushed once and popped once, so the whole thing is ***O(n)*** even though there is a `while` inside the `for`.

Next greater element (values)

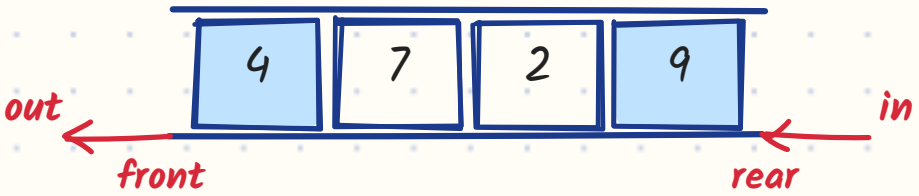
```
def next_greater(a):
    res, st = [-1] * len(a), []
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            res[st.pop()] = x
        st.append(i)
    return res
```

- Also solves: ***largest rectangle in histogram***, ***stock span***, ***trapping rain water***.

QUEUE

Queue & Deque

A ***queue*** is a line at a ticket counter: the ***first*** person in is the ***first*** served — ***FIFO***. Add at the ***rear*** (`enqueue`), remove from the ***front*** (`dequeue`).



New items join the rear; the front leaves first.

```
from collections import deque
q = deque()
q.append(4)             # enqueue
q.append(7)
front = q[0]            # peek
q.popleft()             # dequeue → 4  (O(1))
len(q) == 0             # empty?
```

Python tip
Never use a list as a queue: `list.pop(0)` is O(n) because everything shifts.
Use `collections.deque` → `popleft()` is O(1).

Deque: double-ended queue

A ***deque*** allows adding/removing at ***both ends*** in O(1): `appendleft`, `append`, `popleft`, `pop`. It can behave as a stack ***and*** a queue.

Real-life uses

- Print jobs, CPU scheduling, customer-support lines
- ***BFS*** (breadth-first search) on trees and graphs
- Sliding window maximum (deque)
- Message queues in backend systems

QUEUE

Queue Problems

1. Queue using two stacks

Push onto `inbox`. For `pop`, if `outbox` is empty move everything from inbox (this reverses the order).

```
class MyQueue:
    def __init__(self): self.inb, self.out = [], []
    def push(self, x): self.inb.append(x)
    def pop(self):
        if not self.out:
            while self.inb: self.out.append(self.inb.pop())
        return self.out.pop()
```

Circular queue: store items in a fixed array and wrap indexes with `% size` so freed space is reused.

2. Sliding window maximum (deque)

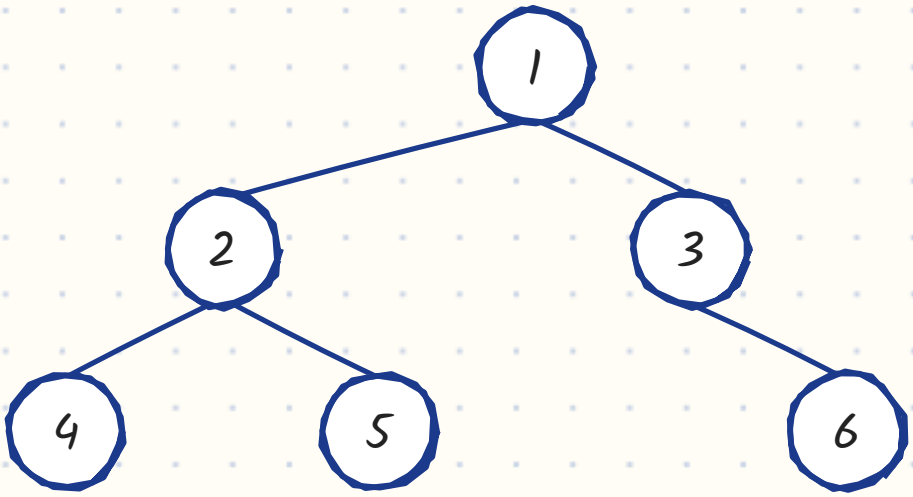
Keep a deque of *indices* whose values are decreasing. The front is always the window's maximum.

```
from collections import deque
def window_max(a, k):
    dq, out = deque(), []
    for i, x in enumerate(a):
        while dq and a[dq[-1]] ⩽ x: dq.pop()
        dq.append(i)
        if dq[0] ⩽ i - k: dq.popleft()
        if i ⩾ k - 1: out.append(a[dq[0]])
    return out
```

TREES

Trees: The Basics

A ***tree*** is a hierarchy: one ***root*** on top, each node can have ***children***. A ***binary tree*** allows at most ***two*** children: left and right. Trees are everywhere: folders, HTML pages, family trees, org charts.



Root = 1. Leaves = 4, 5, 6. Height = 3 levels.

Words to know

Term	Meaning
Root	top node (no parent)
Leaf	node with no children
Depth of a node	edges from the root to it
Height of tree	longest root-to-leaf path
Subtree	a node and all its descendants

tree node in Python

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

root = TreeNode(1,
        TreeNode(2, TreeNode(4), TreeNode(5)),
        TreeNode(3, None, TreeNode(6)))
```

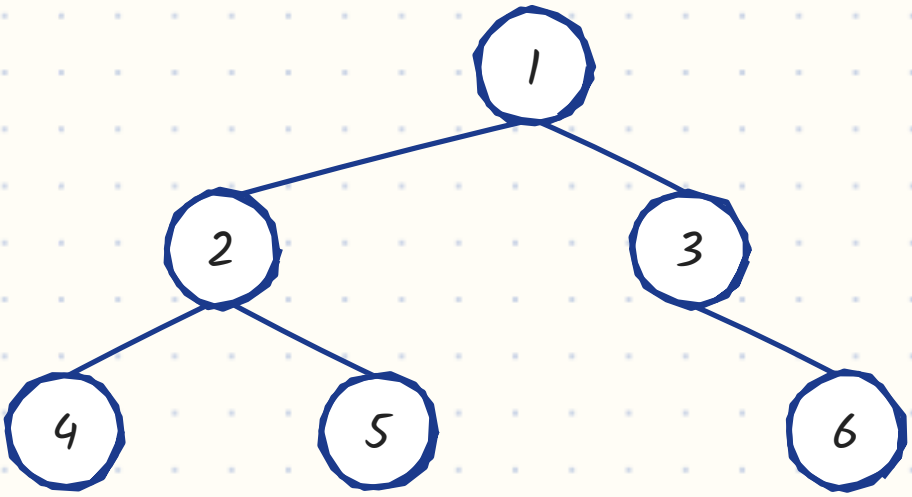
Mindset

Almost every tree problem is solved by ***recursion***: *'solve the left subtree, solve the right subtree, combine the answers'*.

TREES

Tree Traversals (DFS)

Traversal means visiting every node once. Three depth-first orders differ in *when* you visit the current node relative to its children.



Order	Rule	Result for this tree
Preorder	node → left → right	1 2 4 5 3 6
Inorder	left → node → right	4 2 5 1 3 6
Postorder	left → right → node	4 5 2 6 3 1

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def preorder(r):
    return [] if not r else [r.val] + preorder(r.left) + preorder(r.right)
def inorder(r):
    return [] if not r else inorder(r.left) + [r.val] + inorder(r.right)
def postorder(r):
    return [] if not r else postorder(r.left) + postorder(r.right) + [r.val]
```

Remember

Memory trick: the word tells where the **node** goes — **pre** = before children, **in** = in between, **post** = after.

- Preorder: copy a tree · Inorder: gives **sorted** order in a BST · Postorder: delete a tree / compute from the bottom up

TREES

Level Order & Tree Problems

Level order (BFS) — row by row

Use a **queue**. Pop a node, record it, push its children. Process one level at a time using the queue's size.

```
from collections import deque
def level_order(root):
    if not root: return []
    q, out = deque([root]), []
    while q:
        row = []
        for _ in range(len(q)):          # exactly one level
            n = q.popleft(); row.append(n.val)
            if n.left: q.append(n.left)
            if n.right: q.append(n.right)
        out.append(row)
    return out
```

Height / maximum depth

```
def height(r):
    if not r: return 0
    return 1 + max(height(r.left), height(r.right))
```

Count nodes / sum of nodes

```
def count(r):
    return 0 if not r else 1 + count(r.left) + count(r.right)
```

Invert (mirror) a tree

```
def invert(r):
    if r:
        r.left, r.right = invert(r.right), invert(r.left)
    return r
```

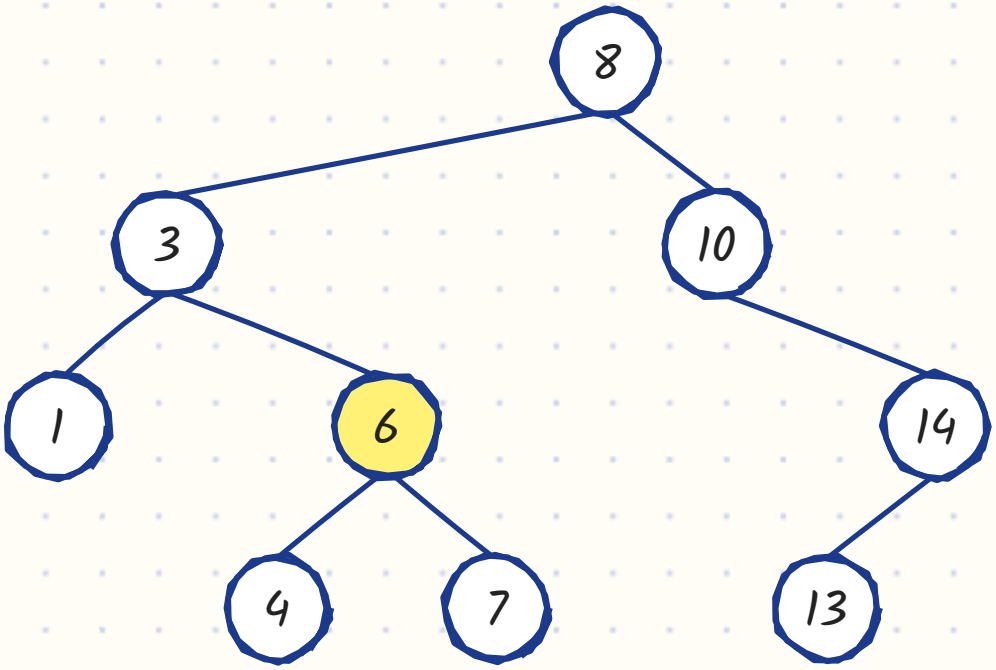
Template

Pattern: **base case** `if not r` → recurse both sides → combine. Practice: same tree? symmetric tree? path sum? diameter?

TREES

Binary Search Tree (BST)

A **BST** keeps order: for every node, everything in the **left** subtree is **smaller** and everything in the **right** is **larger**. Search, insert and delete take **O(h)** (height) — O(log n) if balanced.



Searching 6: 8 → left 3 → right 6. Found in 3 steps.

```
class Node:
    def __init__(self, v): self.val, self.left, self.right = v, None, None

def insert(root, v):
    if not root: return Node(v)
    if v < root.val: root.left = insert(root.left, v)
    else:            root.right = insert(root.right, v)
    return root

def search(root, v):
    while root and root.val ≠ v:
        root = root.left if v < root.val else root.right
    return root
```

Key fact
Inorder traversal of a BST is always sorted. This is used to validate a BST and to find the k-th smallest.

Validate a BST: pass an allowed range `(lo, hi)` down the tree.

TREES

Tree Problems: Diameter, LCA

Diameter of a tree

The longest path between any two nodes. At each node: the path through it = ***left height + right height***. Track the best while computing heights.

```
def diameter(root):
    best = 0
    def h(n):
        nonlocal best
        if not n: return 0
        l, r = h(n.left), h(n.right)
        best = max(best, l + r)
        return 1 + max(l, r)
    h(root)
    return best
```

Lowest common ancestor (LCA)

The lowest node that has both `p` and `q` below it. Recursion: if the current node is `p` or `q`, return it. If both sides return something, the current node is the LCA.

```
def lca(root, p, q):
    if not root or root is p or root is q:
        return root
    L, R = lca(root.left, p, q), lca(root.right, p, q)
    return root if L and R else (L or R)
```

Right side view / path sum

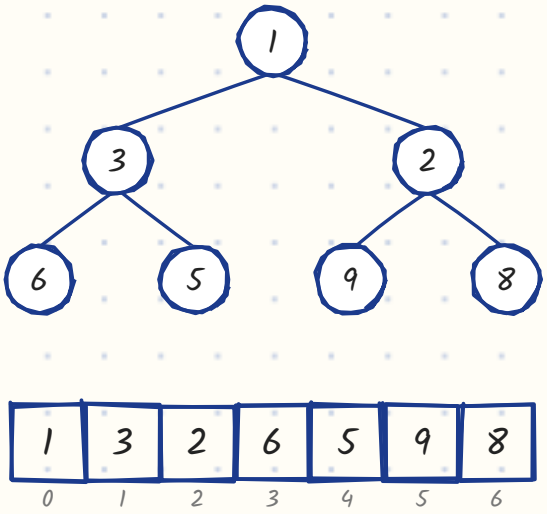
- ***Path sum:*** subtract node value going down; at a leaf check `remaining == 0`.
- ***Right side view:*** last node of every level in level-order.
- ***Same tree:*** compare values, then left, then right.

> ***Encouragement***
> Trees feel hard only until you see the pattern: ***recurse on children, return something useful, combine***. Do 8–10 problems and it clicks.

HEAP

Heap & Priority Queue

A **heap** is a tree stored in an array where every parent is **smaller** than its children (**min-heap**) or larger (**max-heap**). The smallest (or largest) item is always at the **top**, found in O(1). Insert and remove cost **O(log n)**.



Min-heap as a tree (top) and as an array (bottom). Children of index i are 2i+1 and 2i+2.

```
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappush(h, 3)
print(h[0])                  # smallest = 1  (peek)
print(heapq.heappop(h))      # removes 1
heapq.heapify([4, 2, 9])     # list → heap in O(n)

maxh = []                    # max-heap trick: store negatives
heapq.heappush(maxh, -7)
largest = -heapq.heappop(maxh)
```

Python tip
Python's `heapq` is a **min-heap only**. For a max-heap push **negative** numbers, or push tuples like `(-priority, item)`.

Operation	Time
peek min	O(1)
push / pop	O(log n)
build from list (heapify)	O(n)

- Use a heap when you repeatedly need the **smallest / largest / top K** item.
- Used in **Dijkstra**, task schedulers, merging sorted lists, median of a stream.

HEAP

Heap Problems: Top K

1. Kth largest element

Keep a **min-heap of size k**. If it grows beyond k, pop the smallest. At the end the top is the k-th largest. Time **O(n log k)**.

```
import heapq
def kth_largest(a, k):
    h = []
    for x in a:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

2. Top K frequent elements

```
from collections import Counter
import heapq
def top_k(a, k):
    cnt = Counter(a)
    return heapq.nlargest(k, cnt, key=cnt.get)
```

3. K closest points to origin

```
def k_closest(points, k):
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)
```

4. Merge k sorted lists (idea)

Push the first item of every list into a min-heap as `(value, list_id, index)`. Pop the smallest, then push that list's next item. Output is sorted: **O(N log k)**.

5. Running median (idea)

Keep a **max-heap** for the smaller half and a **min-heap** for the larger half. The median is at the tops.

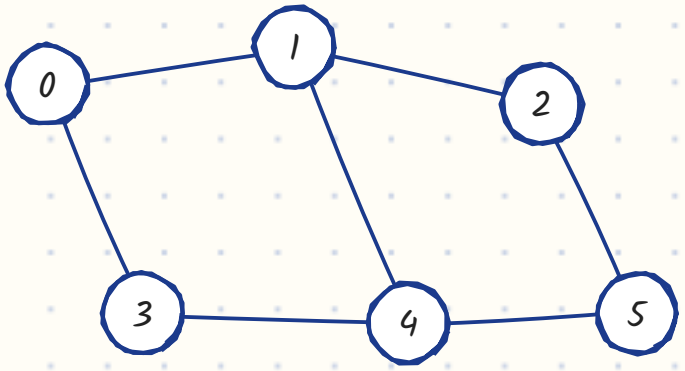
Spot it

Pattern: *'top K', 'K-th', 'K closest'* → think *heap of size K*.

GRAPHS

Graphs: The Basics

A **graph** is a set of **nodes (vertices)** connected by **edges**. It models maps, social networks, web links, flight routes. Edges can be **undirected** (two-way) or **directed** (one-way), and may carry **weights** (distance/cost).



6 nodes, 7 undirected edges

How to store a graph

adjacency list

```
# adjacency list (best!)
g = {
 0: [1, 3],
 1: [0, 2, 4],
 2: [1, 5],
 3: [0, 4],
 4: [1, 3, 5],
 5: [2, 4],
}
```

adjacency matrix

```
# adjacency matrix
#    0 1 2 3 4 5
m = [[0,1,0,1,0,0],
     [1,0,1,0,1,0],
     [0,1,0,0,0,1],
     [1,0,0,0,1,0],
     [0,1,0,1,0,1],
     [0,0,1,0,1,0]]
```

	List	Matrix
Space	O(V+E)	O(V²)
Is edge (u,v)?	O(deg)	O(1)
Best for	sparse graphs	dense graphs

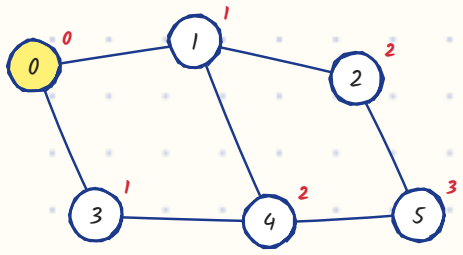
build from an edge list

```
edges = [(0,1),(0,3),(1,2)]
g = {}
for u, v in edges:
    g.setdefault(u, []).append(v)
    g.setdefault(v, []).append(u)   # skip for directed graphs
```

GRAPHS

BFS: Breadth-First Search

BFS explores **level by level**, like ripples in a pond: first all neighbours, then their neighbours. It uses a **queue**. On an unweighted graph, BFS finds the **shortest path** (fewest edges).



Red numbers = distance (levels) from node 0.

```
from collections import deque
def bfs(g, start):
    seen = {start}
    q = deque([start])
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            if v not in seen:       # visit each node once
                seen.add(v)
                q.append(v)
    return order
```

Shortest path (unweighted)

```
def shortest(g, s, t):
    dist = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        if u == t: return dist[u]
        for v in g[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                q.append(v)
    return -1
```

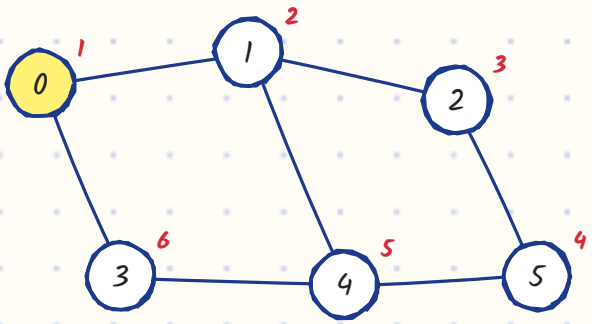
Common bug
Mark a node seen when you push it, not when you pop it.

Time **O(V + E)**. Use for shortest steps, word ladder, rotten oranges.

GRAPHS

DFS: Depth-First Search

DFS goes as **deep** as possible along one path, then **backtracks**. Use recursion (the call stack) or an explicit **stack**. It is perfect for exploring everything, finding connected components and detecting cycles.



One possible DFS order from 0: 0 → 1 → 2 → 5 → 4 → 3

recursive DFS

```
def dfs(g, u, seen=None, order=None):
    if seen is None: seen, order = set(), []
    seen.add(u); order.append(u)
    for v in g[u]:
        if v not in seen:
            dfs(g, v, seen, order)
    return order
```

iterative DFS (stack)

```
def dfs_iter(g, s):
    seen, st, order = set(), [s], []
    while st:
        u = st.pop()
        if u in seen: continue
        seen.add(u); order.append(u)
        for v in reversed(g[u]):
            if v not in seen: st.append(v)
    return order
```

Connected components

```
def components(g):
    seen, count = set(), 0
    for u in g:
        if u not in seen:
            dfs(g, u, seen, [])
            count += 1
    return count
```

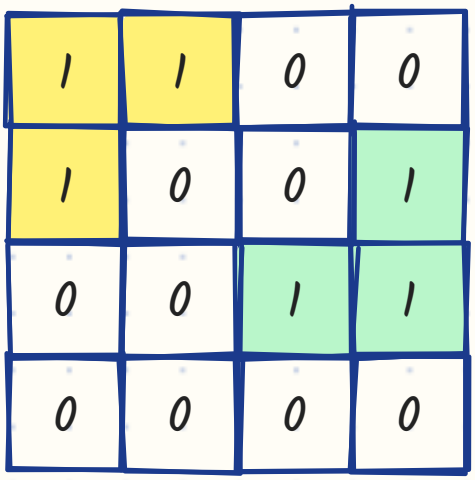
Choose

BFS vs DFS: need the *shortest* path → BFS. Need to *explore/count/search all* → DFS (shorter code).

GRAPHS

Graphs on a Grid

A 2D grid is a hidden graph: each cell is a node, and its **4 neighbours** (up, down, left, right) are edges. Many interview problems are grid DFS/BFS.



'1' = land, '0' = water. This grid has 2 islands.

Number of islands

Scan every cell. When you meet unvisited land, count one island and **flood-fill** (DFS) all connected land to sink it.

```
def num_islands(g):
    R, C = len(g), len(g[0])
    def sink(r, c):
        if not (0 ≤ r < R and 0 ≤ c < C) or g[r][c] ≠ "1":
            return
        g[r][c] = "0"                          # mark visited
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            sink(r + dr, c + dc)
    count = 0
    for r in range(R):
        for c in range(C):
            if g[r][c] == "1":
                count += 1; sink(r, c)
    return count
```

Handy
Direction array trick: `for dr, dc in ((1,0),(-1,0),(0,1),(0,-1))` avoids writing four copies of the same code. Add 4 more pairs for diagonals.

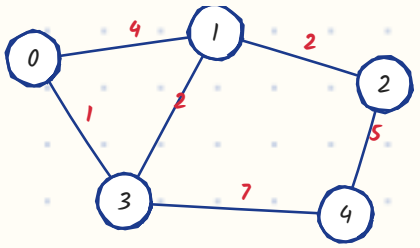
Other grid problems: **rotten oranges** (multi-source BFS), **shortest path in a maze** (BFS), **flood fill**, **surrounded regions**.

GRAPHS

Shortest Paths & Topological Sort

Dijkstra: shortest path with weights

Always expand the **closest unfinished node** using a **min-heap**. Works when all weights are **non-negative**. Time **O((V+E) log V)**.



Shortest 0→2: 0→3→1→2 = 1+2+2 = 5

```
import heapq
def dijkstra(g, src):                # g[u] = [(v, w), ...]
    dist = {src: 0}
    h = [(0, src)]
    while h:
        d, u = heapq.heappop(h)
        if d > dist.get(u, float("inf")): continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(h, (nd, v))
    return dist
```

Topological sort (Kahn's algorithm)

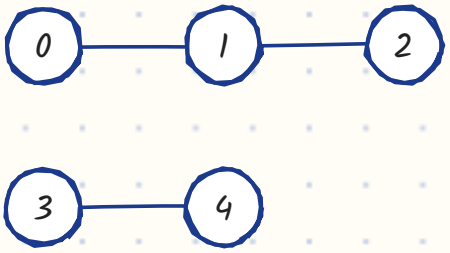
For **directed acyclic graphs** (course prerequisites!): repeatedly take nodes with **in-degree 0**.

```
from collections import deque
def topo(n, edges):
    g = [[] for _ in range(n)]; indeg = [0] * n
    for u, v in edges:
        g[u].append(v); indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    out = []
    while q:
        u = q.popleft(); out.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return out if len(out) == n else []     # [] ⇒ cycle
```

GRAPHS

Union-Find (DSU)

Union-Find (Disjoint Set Union) tracks **groups**. Two operations: `find(x)` → which group is x in? `union(a, b)` → merge two groups. Almost O(1) per operation with two tricks.



Groups: {0,1,2} and {3,4}. Is 2 connected to 4? Compare their group leaders.

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))          # parent of each node
        self.sz = [1] * n
    def find(self, x):
        while self.p[x] ≠ x:
            self.p[x] = self.p[self.p[x]]   # path compression
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False           # already together
        if self.sz[ra] < self.sz[rb]: ra, rb = rb, ra
        self.p[rb] = ra; self.sz[ra] += self.sz[rb]
        return True
```

Why it's fast
Two tricks: *path compression* (flatten the tree while finding) and *union by size* (attach the small tree under the big one).

When to use it

- **Detect a cycle** in an undirected graph: if `union` returns False for an edge, that edge makes a cycle.
- **Count connected components:** start with n, subtract 1 for each successful union.
- **Kruskal's** minimum spanning tree, **accounts merge**, **number of provinces**.

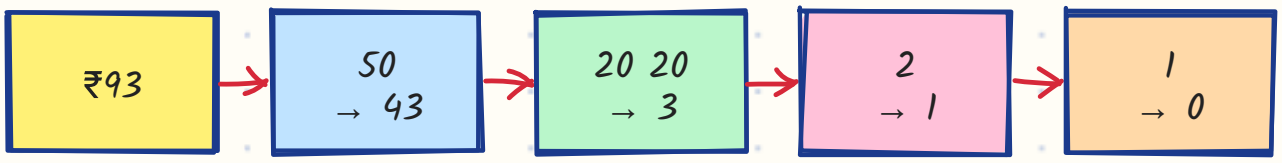
GREEDY

Greedy Algorithms

A **greedy** algorithm makes the **best-looking choice right now**, never looks back, and hopes it leads to the best overall answer. It is fast and simple, but only works when the problem has the **greedy-choice property** — so always justify it.

Example: giving change

Make ₹93 with coins 50, 20, 10, 5, 2, 1. Always take the **biggest coin that fits**.



Greedy picks: 50 + 20 + 20 + 2 + 1 = 5 coins

```
def min_coins(amount, coins=(50, 20, 10, 5, 2, 1)):
    count = 0
    for c in coins:                  # biggest first
        count += amount // c
        amount %= c
    return count
```

Warning

Greedy can fail! With coins {1, 3, 4} and amount 6, greedy gives 4+1+1 = 3 coins, but 3+3 = **2 coins**. When greedy fails, use **DP**.

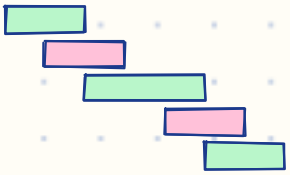
How to recognise greedy

- Words like **maximum, minimum, earliest, fewest**.
- Sorting first usually helps.
- A local best choice never hurts the final answer.

GREEDY

Greedy Problems

1. Max meetings: pick the one that ends earliest



Green = chosen (earliest finish each time).

```
def max_meetings(iv):
    iv.sort(key=lambda x: x[1])      # by end time
    end, count = float("-inf"), 0
    for s, e in iv:
        if s ⩾ end:
            count += 1; end = e
    return count
```

2. Jump game

Track the **farthest reach**; standing beyond it means stuck.

```
def can_jump(a):
    reach = 0
    for i, x in enumerate(a):
        if i > reach: return False
        reach = max(reach, i + x)
    return True
```

3. Best time to buy and sell stock

```
def max_profit(p):
    low, best = float("inf"), 0
    for x in p:
        low = min(low, x)            # cheapest so far
        best = max(best, x - low)    # sell today?
    return best
```

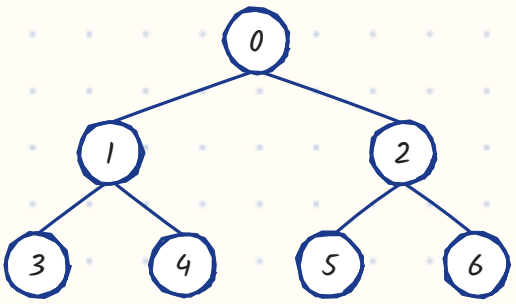
4. Merge overlapping intervals

```
def merge(iv):
    iv.sort()
    out = [iv[0]]
    for s, e in iv[1:]:
        if s ⩽ out[-1][1]: out[-1][1] = max(out[-1][1], e)
        else: out.append([s, e])
    return out
```

BACKTRACKING

Backtracking: Try, Undo, Retry

Backtracking builds a solution one choice at a time. If a choice leads to a dead end, ***undo it*** and try the next. Think of walking a maze: reach a wall, step back, take another turn.



Decision tree: every node is a choice. A dead end sends us back up.

The 3-step template

1. **Choose** an option and add it to the path.
2. **Explore** : recurse with the smaller problem.
3. **Un-choose** : remove it (undo) and try the next option.

```
def backtrack(path, choices):
    if is_solution(path):
        answer.append(path[:])        # copy!
        return
    for c in choices:
        if not valid(c): continue     # prune early
        path.append(c)                # choose
        backtrack(path, next_choices) # explore
        path.pop()                    # un-choose
```

Classic bug
Always save a **copy** `path[:]`. If you save `path` itself, it changes later when you pop.

- **Prune**: stop early when a choice can never work. This makes it fast.
- Good for: ***all subsets, permutations, combinations, puzzles, N-Queens, Sudoku, word search***.

BACKTRACKING

Subsets, Permutations, N-Queens

1. All subsets

For each number: ***take it*** or ***skip it***. n numbers give 2^n subsets.

```
def subsets(a):
    out = []
    def go(i, path):
        if i == len(a):
            out.append(path[:]); return
        path.append(a[i]); go(i + 1, path)   # take
        path.pop();        go(i + 1, path)   # skip
    go(0, [])
    return out
```

2. All permutations

```
def permutations(a):
    out, used = [], [False] * len(a)
    def go(path):
        if len(path) == len(a):
            out.append(path[:]); return
        for i in range(len(a)):
            if used[i]: continue
            used[i] = True; path.append(a[i])
            go(path)
            path.pop(); used[i] = False
    go([])
    return out
```

3. Combination sum

```
def combo_sum(nums, target):
    out = []
    def go(start, path, left):
        if left == 0: out.append(path[:]); return
        for i in range(start, len(nums)):
            if nums[i] > left: continue          # prune
            path.append(nums[i]); go(i, path, left - nums[i]); path.pop()
    go(0, [], target)
    return out
```

4. N-Queens (idea)

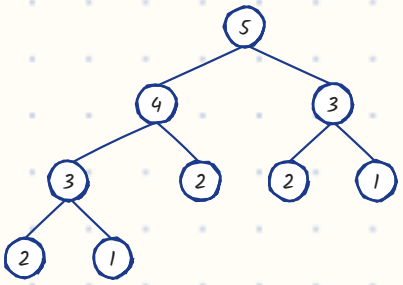
One queen per row; skip any column or diagonal (`r-c` , `r+c`) already used. Dead end? Undo and slide.

DYNAMIC PROGRAMMING

Dynamic Programming: The Idea

DP = recursion + **memory**. If a problem breaks into **overlapping smaller problems**, solve each one **once**, store the answer, and reuse it.

Why plain recursion is slow: Fibonacci



fib(3) and fib(2) are computed again and again. Cost grows like 2^n.

```
from functools import lru_cache
@lru_cache(None)
def fib(n):                        # top-down (memoization)
    return n if n < 2 else fib(n-1) + fib(n-2)

def fib_tab(n):                    # bottom-up (tabulation)
    if n < 2: return n
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Memoization = top-down (recursion + cache). **Tabulation** = bottom-up (loop + table).

5 steps to solve any DP

1. Define **state** : what does `dp[i]` mean?
2. Find the **recurrence** : how does dp[i] come from smaller ones?
3. Set **base cases** .
4. Choose the **order** (small to big).
5. Return the **answer cell** .

Spot DP
Clues: *count ways, min/max cost, can we reach*.

DYNAMIC PROGRAMMING

DP on a Line (1D)

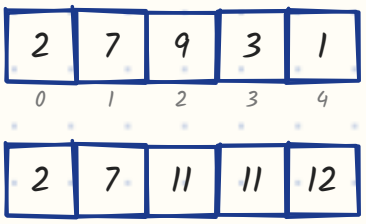
1. Climbing stairs

`dp[i]` = ways to reach step i. You arrive from step i-1 or i-2, so `dp[i] = dp[i-1] + dp[i-2]`.

```
def climb(n):
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

2. House robber

Cannot rob two neighbours. At each house: ***skip it*** (keep best so far) or ***rob it*** (its value + best from two houses back).



Top: money in houses. Bottom: best total so far. Answer 12 (2 + 9 + 1).

```
def rob(a):
    prev, cur = 0, 0
    for x in a:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

3. Coin change (minimum coins)

`dp[x]` = fewest coins for amount x. Try every coin: `dp[x] = 1 + min(dp[x - c])`.

```
def coin_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    for x in range(1, amount + 1):
        for c in coins:
            if c ⩽ x:
                dp[x] = min(dp[x], dp[x - c] + 1)
    return dp[amount] if dp[amount] ≠ INF else -1
```

Greedy gave 3 coins for amount 6 with {1,3,4}. DP finds the true best: 3+3.

DYNAMIC PROGRAMMING

DP on Grids & Strings (2D)

1. Unique paths in a grid

Move only right or down. Ways to reach a cell = ways from above + ways from left.

1	1	1	1
1	2	3	4
1	3	6	10

Each cell = top + left. Bottom-right = 10 paths.

```
def paths(R, C):
    dp = [[1] * C for _ in range(R)]
    for i in range(1, R):
        for j in range(1, C):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[-1][-1]
```

2. Longest common subsequence (LCS)

If the letters match: `1 + dp[i-1][j-1]`. Else: the better of ***skip a letter from A*** or ***from B***.

```
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

3. 0/1 Knapsack

Each item is taken ***once or not at all***. Loop capacity ***backwards*** so an item is not reused.

```
def knapsack(wt, val, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(wt, val):
        for c in range(cap, w - 1, -1):
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]
```

BITS

Bit Manipulation

Computers store numbers in **binary**. Bit tricks are tiny, fast operations that solve neat interview problems.

Operator	Name	Example
&	AND (both 1)	5 & 3 = 1
|	OR (any 1)	5 | 3 = 7
^	XOR (differ)	5 ^ 3 = 6
~	NOT (flip)	~5 = -6
<<	shift left (×2)	3 << 2 = 12
>>	shift right (÷2)	12 >> 2 = 3

Must-know tricks

```
n, x, i = 12, 7, 2
n & 1            # odd? (last bit is 1)
n & (n - 1)      # clears the lowest set bit
n & (n - 1) == 0 # power of two (n > 0)
x ^ x == 0       # a number XOR itself is 0
(n >> i) & 1     # read bit i
n & ~(1 << i)    # clear bit i
```

Single number

Every number appears twice except one. XOR everything: pairs cancel to 0, the lone number remains.

```
def single(a):
    r = 0
    for x in a: r ^= x
    return r
```

Count set bits

```
def count_bits(n):
    c = 0
    while n:
        n &= n - 1; c += 1       # drops one bit each loop
    return c
```

PROBLEM SOLVING

How to Solve Any Coding Problem

Strong problem solvers follow a **routine**, not luck. Use these 7 steps every time.



Say each step out loud in an interview.

1. **Read twice.** Note input, output and limits (n up to what?).
2. **Try small examples** by hand, including an empty input and one element.
3. **Write the brute force** idea first. A correct slow answer beats a wrong fast one.
4. **Optimise:** which data structure removes the repeated work? (hash map, sort, two pointers, heap...)
5. **Code** neatly with clear names.
6. **Test** with your examples and edge cases by tracing the code.
7. **State** time and space complexity.

Read the limits

n is about	Aim for
10	O(n!) or O(2^n) is ok (backtracking)
1,000	O(n^2) is ok
100,000	O(n log n) or O(n)
1,000,000+	O(n) or O(log n)

Best habit
Stuck? **Draw it.** Write a tiny example on paper, watch what you do by hand, then turn those steps into code.

Edge cases checklist

- Empty list / empty string / single element.
- All same values, negative numbers, duplicates.
- Very large input, already sorted or reverse sorted.

PROBLEM SOLVING

Pattern Cheat Sheet: See X, Use Y

Most problems are a **known pattern** in disguise. Match the clue to the tool.

If you see...	Try this	Chapter
Sorted array, find a pair/target	Two pointers or binary search	Arrays, Binary Search
Subarray / substring of size k or longest/shortest	Sliding window	Windows
Range sum queries	Prefix sums	Arrays
Duplicates, counting, 'seen before?'	Hash map / set	Hashing
Brackets, undo, next greater element	Stack / monotonic stack	Stack
Level by level, shortest steps	BFS with a queue	Queue, Graphs
Explore everything, tree/graph paths	DFS / recursion	Trees, Graphs
Top K, K-th largest, smallest	Heap of size K	Heap
Cycle in linked list, middle node	Slow & fast pointers	Linked List
Minimum/maximum value that works	Binary search on the answer	Binary Search
All combinations, subsets, permutations	Backtracking	Backtracking
Count ways, min cost, overlapping subproblems	Dynamic programming	DP
Merge / schedule intervals, earliest finish	Sort + greedy	Greedy
Prerequisites, ordering tasks	Topological sort	Graphs
Groups, connected, same set?	Union-Find	Graphs
Weighted shortest path	Dijkstra	Graphs
Appears twice except one	XOR trick	Bits

Tip

When two patterns fit, start with the simpler one. Brute force → then the pattern that removes the repeated work.

PROBLEM SOLVING

30-Day Practice Plan

Consistency beats cramming. **1 to 2 problems a day**, then review. Always try for 20–30 minutes before looking at a hint.

Days	Topic	Goal problems
1–3	Big-O, arrays, strings	Two sum, reverse array, valid palindrome
4–6	Two pointers, sliding window	Container with most water, longest substring
7–9	Hashing, prefix sums	Group anagrams, subarray sum = k
10–12	Recursion, sorting, binary search	Merge sort, search rotated array
13–15	Linked list, stack, queue	Reverse list, valid parentheses, queue with stacks
16–19	Trees and BST	Level order, max depth, validate BST, LCA
20–22	Heap and graphs	Kth largest, number of islands, course schedule
23–25	Greedy and backtracking	Jump game, subsets, permutations
26–28	Dynamic programming	Climb stairs, house robber, coin change, LCS
29	Mixed revision	Redo the 5 problems you found hardest
30	Mock interview	Solve 2 new problems in 45 minutes, talking aloud

Daily routine (about 1 hour)

1. **10 min:** re-read the pattern from these notes.
2. **30 min:** solve one problem alone.
3. **10 min:** compare with a better solution and note a trick.
4. **10 min:** write the idea in your own words (handwritten!).

> *Secret*
> Keep a **mistakes notebook**. Re-solving a problem you failed is worth three new ones.

PROBLEM SOLVING

Interview Tips & Common Mistakes

During the interview

- **Think aloud.** Interviewers grade your thinking, not just the final code.
- **Clarify** the question: input size, duplicates, negative numbers, sorted or not?
- Start with a **brute force**, state its Big-O, then improve it.
- Write **clean code** with meaningful names and small functions.
- **Dry-run** your code on one example before saying you are done.
- If stuck, ask for a **hint** calmly. That is normal.

Common mistakes

Mistake	Fix
Jumping into code at once	Spend 3 minutes on examples first
Off-by-one errors in loops	Test the first and last index
Forgetting empty input	Check `if not a:` early
Changing a list while looping over it	Loop over a copy or build a new list
Saving `path` instead of `path[:]`	Always copy before storing results
Not marking nodes as visited	Add to `seen` when pushed
Using `==` with floats	Use integers or a small tolerance

Python speed tips

```
from collections import Counter, defaultdict, deque
a = [3, 1, 2]
Counter("banana")              # {'a': 3, 'n': 2, 'b': 1}
defaultdict(list)              # missing key → empty list
deque([1, 2, 3])               # fast pop from both ends
sorted(a, key=lambda x: -x)    # sort by custom rule
```

You can do it

Interviews test **patterns you have practised**. You already have the toolbox. Now repeat, repeat, repeat.

PROBLEM SOLVING

References & Further Learning

These notes were written in my own words with **original hand-drawn diagrams**. Every code sample was run and tested. Resources below helped plan the order.

Roadmaps and guides

- **Scaler:** DSA roadmap for beginners: https://www.scaler.com/blog/dsa-roadmap/
- **WsCubeTech:** DSA roadmap: https://www.wscubetech.com/blog/dsa-roadmap/
- **roadmap.sh:** Data Structures & Algorithms path: https://roadmap.sh/datastructures-and-algorithms

See algorithms move (animations)

- **VisuAlgo:** https://visualgo.net/ (sorting, trees, graphs, heaps, hashing)
- **USFCA Data Structure Visualizations:** https://www.cs.usfca.edu/~galles/visualization/

Read and learn

- **GeeksforGeeks:** https://www.geeksforgeeks.org/data-structures/ (concept articles)
- **Python docs:** https://docs.python.org/3/library/ (collections, heapq, bisect)
- **CLRS:** *Introduction to Algorithms* (Cormen et al.) for deep theory
- **Grokking Algorithms** by Aditya Bhargava: the friendliest beginner book

Practice problems

- **LeetCode:** https://leetcode.com/ (start with Easy tagged problems)
- **NeetCode 150:** https://neetcode.io/ (pattern-wise list with videos)
- **HackerRank:** https://www.hackerrank.com/domains/data-structures

More from us
Visit the **PrepPath E-books Store** website to explore more e-books for your learning journey. Shop now!